Personal Sleep Performance Optimization Using Data Analytics

IMPORTS AND SETUP

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
from datetime import datetime, timedelta
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Statistical testing
from scipy import stats
from scipy.stats import pearsonr, spearmanr, ttest_ind, chi2_contingency
import statsmodels.api as sm
from statsmodels.stats.proportion import proportions_ztest

# Machine Learning (for correlation and feature selection)
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans

# Configure display settings
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 100)
plt.style.use('seaborn-v0_8')
sns.set_palette("husl")
warnings.filterwarnings('ignore')
```

DATA LOADING AND INITIAL INSPECTION

In [ ]:
```
# Load the dataset
df_raw = pd.read_csv('com.samsung.shealth.sleep.20250808163269.csv')

print(f"Dataset shape: {df_raw.shape}")
print(f"Memory usage: {df_raw.memory_usage(deep=True).sum() / 1024**2:.2f} MB")
```

```
Dataset shape: (818, 61)
Memory usage: 0.87 MB
```

In [ ]:
```
# Display basic info
print("\nDataset Info:")
df_raw.info()

print("\nFirst 3 rows:")
print(df_raw.head(3))
```

```
Dataset Info:
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 818 entries, 0 to 817
Data columns (total 61 columns):
 #   Column                                      Non-Null Count  Dtype
---  ------                                      --------------  -----
 0   total_sleep_time_weight                     69 non-null     float64
 1   original_efficiency                         21 non-null     float64
 2   mental_recovery                             811 non-null    float64
 3   wake_score                                  69 non-null     float64
 4   factor_01                                   811 non-null    float64
 5   factor_02                                   811 non-null    float64
 6   factor_03                                   811 non-null    float64
 7   factor_04                                   811 non-null    float64
 8   factor_05                                   811 non-null    float64
 9   factor_06                                   811 non-null    float64
 10  factor_07                                   811 non-null    float64
 11  factor_08                                   811 non-null    float64
 12  factor_09                                   811 non-null    float64
 13  factor_10                                   377 non-null    float64
 14  deep_score                                  69 non-null     float64
 15  integrated_id                               0 non-null      float64
 16  latency_weight                              69 non-null     float64
 17  has_sleep_data                              818 non-null    int64
 18  bedtime_detection_delay                     420 non-null    float64
 19  sleep_efficiency_with_latency               69 non-null     float64
 20  wakeup_time_detection_delay                 420 non-null    float64
 21  total_rem_duration                          434 non-null    float64
 22  combined_id                                 126 non-null    object
 23  sleep_type                                  812 non-null    float64
 24  sleep_latency                               420 non-null    float64
 25  data_version                                813 non-null    float64
 26  latency_score                               69 non-null     float64
 27  deep_weight                                 69 non-null     float64
 28  rem_weight                                  69 non-null     float64
 29  physical_recovery                           811 non-null    float64
 30  original_wake_up_time                       459 non-null    object
 31  movement_awakening                          811 non-null    float64
 32  is_integrated                               0 non-null      float64
 33  original_bed_time                           459 non-null    object
 34  goal_bed_time                               224 non-null    float64
 35  quality                                     3 non-null      float64
 36  extra_data                                  379 non-null    object
 37  wake_weight                                 69 non-null     float64
 38  rem_score                                   69 non-null     float64
 39  goal_wake_up_time                           224 non-null    float64
 40  sleep_cycle                                 811 non-null    float64
 41  total_light_duration                        434 non-null    float64
 42  efficiency                                  816 non-null    float64
 43  sleep_score                                 811 non-null    float64
 44  sleep_duration                              811 non-null    float64
 45  stage_analyzed_type                         434 non-null    float64
 46  total_sleep_time_score                      69 non-null     float64
 47  com.samsung.health.sleep.create_sh_ver      650 non-null    float64
 48  com.samsung.health.sleep.start_time         818 non-null    object
 49  com.samsung.health.sleep.custom             0 non-null      float64
 50  com.samsung.health.sleep.modify_sh_ver      650 non-null    float64
 51  com.samsung.health.sleep.update_time        818 non-null    object
 52  com.samsung.health.sleep.create_time        818 non-null    object
 53  com.samsung.health.sleep.client_data_id     0 non-null      float64
 54  com.samsung.health.sleep.client_data_ver    0 non-null      float64
 55  com.samsung.health.sleep.time_offset        818 non-null    object
 56  com.samsung.health.sleep.deviceuuid         818 non-null    object
 57  com.samsung.health.sleep.comment            0 non-null      float64
 58  com.samsung.health.sleep.pkg_name           818 non-null    object
 59  com.samsung.health.sleep.end_time           818 non-null    object
 60  com.samsung.health.sleep.datauuid           818 non-null    object
dtypes: float64(48), int64(1), object(12)
memory usage: 390.0+ KB

First 3 rows:
   total_sleep_time_weight  original_efficiency  mental_recovery  wake_score  \
0                      NaN                 NaN             57.0         NaN
1                      NaN                 NaN             37.0         NaN
2                      NaN                 NaN             41.0         NaN

   factor_01  factor_02  factor_03  factor_04  factor_05  factor_06  \
0       17.0       24.0       29.0       23.0       86.0      495.0
1       20.0       41.0       14.0       18.0       77.0      519.0
2        7.0       20.0        6.0       24.0       70.0      453.0

   factor_07  factor_08  factor_09  factor_10  deep_score  integrated_id  \
0      178.0        1.0       41.0        0.0         NaN            NaN
1      188.0        3.0       45.0        0.0         NaN            NaN
```

```
2       66.0        3.0       40.0        0.0         NaN            NaN

   latency_weight  has_sleep_data  bedtime_detection_delay  \
0             NaN               0                      NaN
1             NaN               0                      NaN
2             NaN               0                      NaN

   sleep_efficiency_with_latency  wakeup_time_detection_delay  \
0                            NaN                          NaN
1                            NaN                          NaN
2                            NaN                          NaN

   total_rem_duration combined_id  sleep_type  sleep_latency  data_version  \
0                 NaN         NaN         5.0            NaN           3.0
1                 NaN         NaN        -1.0            NaN           3.0
2                 NaN         NaN        -1.0            NaN           3.0

   latency_score  deep_weight  rem_weight  physical_recovery  \
0            NaN          NaN         NaN               58.0
1            NaN          NaN         NaN               64.0
2            NaN          NaN         NaN               54.0

  original_wake_up_time  movement_awakening  is_integrated original_bed_time  \
0                   NaN                42.0            NaN               NaN
1                   NaN                31.0            NaN               NaN
2                   NaN                33.0            NaN               NaN

   goal_bed_time  quality                                         extra_data  \
0            NaN      NaN  760f93fc-c63c-401c-9001-82ddb08ee706.extra_dat...
1            NaN      NaN  63def948-a231-4c41-873d-94ffc6b18552.extra_dat...
2            NaN      NaN  ba3a3aa5-233c-4c3c-967c-8b65775a7a8f.extra_dat...

   wake_weight  rem_score  goal_wake_up_time  sleep_cycle  \
0          NaN        NaN                NaN          4.0
1          NaN        NaN                NaN          3.0
2          NaN        NaN                NaN          5.0

   total_light_duration  efficiency  sleep_score  sleep_duration  \
0                   NaN        82.0         59.0           495.0
1                   NaN        85.0         58.0           519.0
2                   NaN        84.0         63.0           453.0

   stage_analyzed_type  total_sleep_time_score  \
0                  NaN                     NaN
1                  NaN                     NaN
2                  NaN                     NaN

   com.samsung.health.sleep.create_sh_ver com.samsung.health.sleep.start_time  \
0                                     NaN                    26-08-2023 18:06
1                                     NaN                    14-08-2023 17:42
2                                     NaN                    15-08-2023 17:39

   com.samsung.health.sleep.custom  com.samsung.health.sleep.modify_sh_ver  \
0                              NaN                                      NaN
1                              NaN                                      NaN
2                              NaN                                      NaN

  com.samsung.health.sleep.update_time com.samsung.health.sleep.create_time  \
0                    27-08-2023 02:27                     27-08-2023 02:27
1                    15-08-2023 02:25                     15-08-2023 02:24
2                    16-08-2023 01:15                     16-08-2023 01:12

   com.samsung.health.sleep.client_data_id  \
0                                      NaN
1                                      NaN
2                                      NaN

   com.samsung.health.sleep.client_data_ver  \
0                                       NaN
1                                       NaN
2                                       NaN

  com.samsung.health.sleep.time_offset com.samsung.health.sleep.deviceuuid  \
0                            UTC+0530                          Z5gbspmoTN
1                            UTC+0530                          Z5gbspmoTN
2                            UTC+0530                          Z5gbspmoTN

   com.samsung.health.sleep.comment com.samsung.health.sleep.pkg_name  \
0                               NaN       com.sec.android.app.shealth
1                               NaN       com.sec.android.app.shealth
2                               NaN       com.sec.android.app.shealth

  com.samsung.health.sleep.end_time     com.samsung.health.sleep.datauuid
```

```
0                           21:00.0  760f93fc-c63c-401c-9001-82ddb08ee706
1                           21:00.0  63def948-a231-4c41-873d-94ffc6b18552
2                           12:00.0  ba3a3aa5-233c-4c3c-967c-8b65775a7a8f
```

- Nights recorded: **818** (roughly a bit over 2 years, assuming near-daily tracking).
- Columns: **61**, which is a lot of mixed metadata and derived metrics from Samsung Health.
- Memory footprint is only **~0.87 MB**, so everything is lightweight and easy to analyze on a laptop.
- This is plenty of data to detect patterns in sleep behavior over time.

DATA QUALITY ASSESSMENT

In [ ]:
```python
# Missing values analysis
missing_data = pd.DataFrame({
    'Column': df_raw.columns,
    'Missing_Count': df_raw.isnull().sum(),
    'Missing_Percentage': (df_raw.isnull().sum()/len(df_raw)) * 100,
    'Data_Type': df_raw.dtypes
}).sort_values('Missing_Percentage', ascending=False)

print("Missing Values Analysis:")
print(missing_data.head(15))
```

```
Missing Values Analysis:
                                                                    Column  \
integrated_id                                                  integrated_id
com.samsung.health.sleep.custom                     com.samsung.health.sleep.custom
com.samsung.health.sleep.comment                   com.samsung.health.sleep.comment
com.samsung.health.sleep.client_data_ver  com.samsung.health.sleep.client_data_ver
is_integrated                                                  is_integrated
com.samsung.health.sleep.client_data_id    com.samsung.health.sleep.client_data_id
quality                                                                quality
original_efficiency                                       original_efficiency
deep_weight                                                         deep_weight
sleep_efficiency_with_latency                       sleep_efficiency_with_latency
total_sleep_time_weight                                   total_sleep_time_weight
deep_score                                                           deep_score
wake_score                                                           wake_score
wake_weight                                                         wake_weight
rem_score                                                             rem_score

                                          Missing_Count  Missing_Percentage  \
integrated_id                                       818          100.000000
com.samsung.health.sleep.custom                     818          100.000000
com.samsung.health.sleep.comment                    818          100.000000
com.samsung.health.sleep.client_data_ver            818          100.000000
is_integrated                                       818          100.000000
com.samsung.health.sleep.client_data_id             818          100.000000
quality                                             815           99.633252
original_efficiency                                 797           97.432763
deep_weight                                         749           91.564792
sleep_efficiency_with_latency                       749           91.564792
total_sleep_time_weight                             749           91.564792
deep_score                                          749           91.564792
wake_score                                          749           91.564792
wake_weight                                         749           91.564792
rem_score                                           749           91.564792

                                         Data_Type
integrated_id                              float64
com.samsung.health.sleep.custom            float64
com.samsung.health.sleep.comment           float64
com.samsung.health.sleep.client_data_ver   float64
is_integrated                              float64
com.samsung.health.sleep.client_data_id    float64
quality                                    float64
original_efficiency                        float64
deep_weight                                float64
sleep_efficiency_with_latency              float64
total_sleep_time_weight                    float64
deep_score                                 float64
wake_score                                 float64
wake_weight                                float64
rem_score                                  float64
```

- The dataset contains a mix of:
  - Identifiers and integration flags (`integrated_id` , `is_integrated` , client IDs).
  - Core sleep metrics (duration, score, efficiency, stages).
  - Extra Samsung-specific factors and metadata columns.

- Some columns are non-numeric / object types that will need:
  - Parsing (e.g., timestamps).
  - Dropping if they're pure metadata or unusable for analysis.
- The info output confirms that the CSV was read correctly and that the column types look consistent.

In [ ]:
```
# Duplicate analysis
duplicates = df_raw.duplicated().sum()
print(f"\nDuplicate rows: {duplicates}")

# Special check for sleep stage data availability
sleep_stage_cols = ['total_rem_duration', 'total_light_duration']
for col in sleep_stage_cols:
    if col in df_raw.columns:
        print(f"\n{col} availability:")
        print(f"  Total missing: {df_raw[col].isnull().sum()}")
        print(f"  Missing percentage: {df_raw[col].isnull().sum() / len(df_raw) * 100:.1f}%")
```

```
Duplicate rows: 0

total_rem_duration availability:
  Total missing: 384
  Missing percentage: 46.9%

total_light_duration availability:
  Total missing: 384
  Missing percentage: 46.9%
```

DATA CLEANING AND PREPROCESSING

In [ ]:
```
# Define the columns we want to keep and their new names
target_columns = {
    'com.samsung.health.sleep.start_time': 'sleep_start_time',
    'sleep_duration': 'sleep_duration_minutes',
    'sleep_score': 'overall_sleep_score',
    'efficiency': 'sleep_efficiency_percent',
    'sleep_cycle': 'sleep_cycle_count',
    'mental_recovery': 'rem_sleep_percentage',
    'physical_recovery': 'deep_sleep_percentage',
    'factor_05': 'total_awake_duration',
    'total_rem_duration': 'total_rem_minutes',
    'total_light_duration': 'total_light_minutes',
    'factor_02': 'total_deep_minutes',
    'movement_awakening': 'awake_percentage'
}

# Check which columns exist in the dataset
available_columns = {}
missing_columns = []

for old_name, new_name in target_columns.items():
    if old_name in df_raw.columns:
        available_columns[old_name] = new_name
    else:
        missing_columns.append(old_name)

print(f"Available columns: {len(available_columns)}")
print(f"Missing columns: {len(missing_columns)}")

if missing_columns:
    print("Missing columns:")
    for col in missing_columns:
        print(f"  - {col}")

# Create cleaned dataset with only available target columns
df = df_raw[list(available_columns.keys())].copy()
df = df.rename(columns=available_columns)

print(f"\nDataset after column selection: {df.shape}")
```

```
Available columns: 12
Missing columns: 0

Dataset after column selection: (818, 12)
```

- A large chunk of missingness is concentrated in:
  - Integration / client metadata columns.
  - Custom / comment fields.
- These are not critical for sleep behavior analysis.
- Core metrics (duration, score, efficiency, etc.) have **much lower missingness**, which is what matters.

- This justifies focusing on a smaller subset of columns instead of trying to salvage every raw field.

Handling datetime

In [ ]:
```
# Handle datetime conversion
if 'sleep_start_time' in df.columns:
    try:
        df['sleep_start_time'] = pd.to_datetime(df['sleep_start_time'], errors='coerce')

        # Convert from GMT to IST by adding 5 hours 30 minutes
        df['sleep_start_time'] = df['sleep_start_time'] + pd.Timedelta(hours=5, minutes=30)

        df['sleep_date'] = df['sleep_start_time'].dt.date
        print("Successfully converted sleep_start_time to datetime")
    except Exception as e:
        print(f"Failed to convert sleep_start_time: {e}")


# CRITICAL: Handle sleep stage data availability
# Create flags to track which data is available for which analyses
print("\nCREATING DATA AVAILABILITY FLAGS:")

if 'sleep_date' in df.columns:
    df['sleep_date_dt'] = pd.to_datetime(df['sleep_date'])

    # Flag for sleep stage data availability
    sleep_stage_columns = ['total_rem_minutes', 'total_light_minutes']
    available_stage_cols = [col for col in sleep_stage_columns if col in df.columns]

    if available_stage_cols:
        # Check if both REM and Light data are present
        df['has_complete_stage_data'] = df[available_stage_cols].notna().all(axis=1)

        complete_data_count = df['has_complete_stage_data'].sum()
        total_records = len(df)

        print(f"  - Complete sleep stage data available for: {complete_data_count}/{total_records} nights")
        print(f"  - Sleep stage data coverage: {complete_data_count/total_records*100:.1f}%")

        # Identify the period with complete data
        if complete_data_count > 0:
            complete_data_df = df[df['has_complete_stage_data']]
            stage_data_start = complete_data_df['sleep_date'].min()
            stage_data_end = complete_data_df['sleep_date'].max()
            print(f"  - Sleep stage data period: {stage_data_start} to {stage_data_end}")
```

```
Successfully converted sleep_start_time to datetime

CREATING DATA AVAILABILITY FLAGS:
  - Complete sleep stage data available for: 434/818 nights
  - Sleep stage data coverage: 53.1%
  - Sleep stage data period: 2024-07-05 to 2025-08-08
```

In [ ]:
```
# Basic metrics that should be available across all periods
basic_metrics = ['overall_sleep_score', 'sleep_duration_minutes', 'sleep_efficiency_percent']
df['has_basic_metrics'] = df[basic_metrics].notna().all(axis=1)

basic_data_count = df['has_basic_metrics'].sum()
print(f"  - Complete basic metrics available for: {basic_data_count}/{len(df)} nights")

# Handle missing values with temporal awareness
numeric_columns = df.select_dtypes(include=[np.number]).columns

print(f"\nHandling missing values in numeric columns...")
for col in numeric_columns:
    if df[col].isnull().sum() > 0:
        # For sleep stage data, keep as NaN if systematically missing
        if col in ['total_rem_minutes', 'total_light_minutes']:
            print(f"Keeping {col} NaN values (systematic missing data) - {df[col].isnull().sum()} missing")
        else:
            # Use median imputation for other metrics
            median_val = df[col].median()
            df[col].fillna(median_val, inplace=True)
            print(f"Filled {col} missing values with median: {median_val:.2f}")

print(f"\nCleaned dataset shape: {df.shape}")
```

```
  - Complete basic metrics available for: 811/818 nights

Handling missing values in numeric columns...
Filled sleep_duration_minutes missing values with median: 454.00
Filled overall_sleep_score missing values with median: 76.00
Filled sleep_efficiency_percent missing values with median: 86.00
Filled sleep_cycle_count missing values with median: 4.00
Filled rem_sleep_percentage missing values with median: 76.00
Filled deep_sleep_percentage missing values with median: 66.00
Filled total_awake_duration missing values with median: 59.00
Keeping total_rem_minutes NaN values (systematic missing data) - 384 missing
Keeping total_light_minutes NaN values (systematic missing data) - 384 missing
Filled total_deep_minutes missing values with median: 42.00
Filled awake_percentage missing values with median: 30.00

Cleaned dataset shape: (818, 16)
```

- `sleep_start_time` has been:
  - Parsed as a datetime.
  - Shifted from GMT to **IST (+5:30)** → timestamps now match your local timezone.
- Sleep stage completeness:
  - **434 / 818 nights** have full stage data (**~53.1%** coverage).
  - These nights range from **2024-07-05** to **2025-08-08**.
- You now have two clear regimes:
  - **Full-metric period:** all basic metrics + full sleep stage breakdown (post 2024-07-05).
  - **Basic-only period:** earlier nights with limited or no stage info.
- Any insight using stages (REM/light/deep) must be explicitly framed as “based on the 434 fully-tracked nights”.

FEATURE ENGINEERING

In [ ]:
```python
# Convert sleep duration to hours
if 'sleep_duration_minutes' in df.columns:
    df['sleep_duration_hours'] = df['sleep_duration_minutes'] / 60
    print("Created sleep_duration_hours")

# Calculate sleep stage percentages ONLY when data is available
print("\nHandling sleep stage calculations:")
if all(col in df.columns for col in ['total_rem_minutes', 'total_light_minutes', 'total_deep_minutes', 'sleep_d
    # Only calculate for rows where stage data exists
    stage_mask = df['has_complete_stage_data']

    if stage_mask.any():
        # Calculate percentages only for complete data
        df.loc[stage_mask, 'rem_sleep_pct_calculated'] = (
            df.loc[stage_mask, 'total_rem_minutes'] / df.loc[stage_mask, 'sleep_duration_minutes'] * 100
        ).clip(0, 100)

        df.loc[stage_mask, 'light_sleep_pct_calculated'] = (
            df.loc[stage_mask, 'total_light_minutes'] / df.loc[stage_mask, 'sleep_duration_minutes'] * 100
        ).clip(0, 100)

        df.loc[stage_mask, 'deep_sleep_pct_calculated'] = (
            df.loc[stage_mask, 'total_deep_minutes'] / df.loc[stage_mask, 'sleep_duration_minutes'] * 100
        ).clip(0, 100)

        print(f"Created calculated sleep stage percentages for {stage_mask.sum()} nights with complete data")
        print("Sleep stage percentages left as NaN for incomplete data periods")
    else:
        print("No complete sleep stage data found - skipping stage percentage calculations")
```

```
Created sleep_duration_hours

Handling sleep stage calculations:
Created calculated sleep stage percentages for 434 nights with complete data
Sleep stage percentages left as NaN for incomplete data periods
```

In [ ]:
```python
# Extract temporal features from sleep start time
if 'sleep_start_time' in df.columns:
    df['bedtime_hour'] = df['sleep_start_time'].dt.hour
    df['bedtime_day_of_week'] = df['sleep_start_time'].dt.day_name()
    df['bedtime_weekday_num'] = df['sleep_start_time'].dt.weekday  # 0=Monday, 6=Sunday
    df['is_weekend'] = df['bedtime_weekday_num'].isin([5, 6])  # Saturday and Sunday

    # Create bedtime categories
    df['bedtime_category'] = pd.cut(df['bedtime_hour'],
                                    bins=[-1, 2, 6, 22, 24, 25],
                                    labels=['Late Night', 'Very Late', 'Early Evening', 'Normal Bedtime', 'Early
                                    ordered=False)
```

```python
    print("Created temporal features from bedtime")

# Create sleep quality categories
if 'overall_sleep_score' in df.columns:
    df['sleep_quality_category'] = pd.cut(df['overall_sleep_score'],
                                          bins=[0, 60, 75, 85, 100],
                                          labels=['Poor', 'Fair', 'Good', 'Excellent'],
                                          ordered=True)
    print("Created sleep quality categories")

# Create efficiency categories
if 'sleep_efficiency_percent' in df.columns:
    df['efficiency_category'] = pd.cut(df['sleep_efficiency_percent'],
                                       bins=[0, 80, 90, 95, 100],
                                       labels=['Low', 'Average', 'High', 'Excellent'],
                                       ordered=True)
    print("Created efficiency categories")

# Calculate sleep consistency metrics (7-day rolling window)
if 'sleep_date' in df.columns and len(df) > 7:
    df = df.sort_values('sleep_date')

    if 'overall_sleep_score' in df.columns:
        df['sleep_score_7day_avg'] = df['overall_sleep_score'].rolling(window=7, min_periods=1).mean()
        df['sleep_score_consistency'] = df['overall_sleep_score'].rolling(window=7, min_periods=1).std()

    if 'bedtime_hour' in df.columns:
        df['bedtime_consistency'] = df['bedtime_hour'].rolling(window=7, min_periods=1).std()

    print("Created sleep consistency metrics")

# Sleep debt calculation (assuming 8 hours is optimal)
if 'sleep_duration_hours' in df.columns:
    optimal_sleep = 8.0
    df['sleep_debt'] = optimal_sleep - df['sleep_duration_hours']
    df['is_sleep_debt'] = df['sleep_debt'] > 0
    df['is_oversleep'] = df['sleep_debt'] < -0.5  # More than 30 minutes over 8 hours
    print("Created sleep debt metrics")

print(f"\nFinal dataset shape: {df.shape}")
```

```
Created temporal features from bedtime
Created sleep quality categories
Created efficiency categories
Created sleep consistency metrics
Created sleep debt metrics

Final dataset shape: (818, 33)
```

- Temporal features added:
  - `bedtime_hour`, `bedtime_day_of_week`, `is_weekend`, etc.
  - `bedtime_category` buckets nights into interpretable time bands (Late Night, Very Late, Normal, etc.).
- Quality and efficiency:
  - `sleep_quality_category` and `efficiency_category` convert raw scores into **discrete quality bands** (e.g., Poor / Average / Good).
- Consistency & debt:
  - Rolling metrics like `sleep_score_7day_avg`, `sleep_score_consistency`, and `bedtime_consistency` quantify **stability** over time.
  - `sleep_debt`, `is_sleep_debt`, `is_oversleep` put a numeric value on how often you are **under-sleeping or oversleeping** relative to an 8-hour target.
- Final engineered dataset shape: **(818, 33)** → rich enough for correlation analysis, clustering, and time-series storytelling.

In [ ]:
```python
print(df.head())
```

```
      sleep_start_time  sleep_duration_minutes  overall_sleep_score  \
5  2023-08-05 23:32:00                   521.0                 72.0
17 2023-08-06 23:34:00                   403.0                 66.0
33 2023-08-07 23:04:00                   433.0                 66.0
46 2023-08-08 22:55:00                   498.0                 66.0
55 2023-08-09 23:05:00                   445.0                 75.0

    sleep_efficiency_percent  sleep_cycle_count  rem_sleep_percentage  \
5                       84.0                3.0                  81.0
17                      85.0                2.0                  61.0
33                      80.0                3.0                  67.0
46                      75.0                3.0                  65.0
55                      88.0                3.0                  40.0

    deep_sleep_percentage  total_awake_duration  total_rem_minutes  \
5                    58.0                  79.0                NaN
17                   59.0                  58.0                NaN
33                   58.0                  85.0                NaN
46                   61.0                 121.0                NaN
55                   67.0                  49.0                NaN

    total_light_minutes  total_deep_minutes  awake_percentage  sleep_date  \
5                   NaN                25.0              36.0  2023-08-05
17                  NaN                35.0              28.0  2023-08-06
33                  NaN                38.0              52.0  2023-08-07
46                  NaN                33.0              69.0  2023-08-08
55                  NaN                38.0              18.0  2023-08-09

   sleep_date_dt  has_complete_stage_data  has_basic_metrics  \
5     2023-08-05                    False               True
17    2023-08-06                    False               True
33    2023-08-07                    False               True
46    2023-08-08                    False               True
55    2023-08-09                    False               True

    sleep_duration_hours  rem_sleep_pct_calculated  \
5               8.683333                       NaN
17              6.716667                       NaN
33              7.216667                       NaN
46              8.300000                       NaN
55              7.416667                       NaN

    light_sleep_pct_calculated  deep_sleep_pct_calculated  bedtime_hour  \
5                          NaN                        NaN            23
17                         NaN                        NaN            23
33                         NaN                        NaN            23
46                         NaN                        NaN            22
55                         NaN                        NaN            23

   bedtime_day_of_week  bedtime_weekday_num  is_weekend bedtime_category  \
5             Saturday                    5        True   Normal Bedtime
17              Sunday                    6        True   Normal Bedtime
33              Monday                    0       False   Normal Bedtime
46             Tuesday                    1       False    Early Evening
55           Wednesday                    2       False   Normal Bedtime

   sleep_quality_category efficiency_category  sleep_score_7day_avg  \
5                    Fair             Average                  72.0
17                   Fair             Average                  69.0
33                   Fair                 Low                  68.0
46                   Fair                 Low                  67.5
55                   Fair             Average                  69.0

    sleep_score_consistency  bedtime_consistency  sleep_debt  is_sleep_debt  \
5                       NaN                  NaN   -0.683333          False
17                 4.242641             0.000000    1.283333           True
33                 3.464102             0.000000    0.783333           True
46                 3.000000             0.500000   -0.300000          False
55                 4.242641             0.447214    0.583333           True

    is_oversleep
5           True
17         False
33         False
46         False
55         False
```

- The sample rows confirm:
  - `sleep_start_time` is in a proper datetime format.
  - `sleep_duration_minutes`, `overall_sleep_score`, and `sleep_efficiency_percent` have reasonable values (no obvious junk).
  - Newly engineered columns like `sleep_cycle_count`, `rem_sleep_percentage`, and `deep_sleep_percentage` look

numeric and interpretable.

- This is a quick visual verification that the transformations didn't introduce obvious corruption or misalignment.

In [ ]:
```
# Check the columns in the data

df.columns
```

Out[ ]:
```
Index(['sleep_start_time', 'sleep_duration_minutes', 'overall_sleep_score',
       'sleep_efficiency_percent', 'sleep_cycle_count', 'rem_sleep_percentage',
       'deep_sleep_percentage', 'total_awake_duration', 'total_rem_minutes',
       'total_light_minutes', 'total_deep_minutes', 'awake_percentage',
       'sleep_date', 'sleep_date_dt', 'has_complete_stage_data',
       'has_basic_metrics', 'sleep_duration_hours', 'rem_sleep_pct_calculated',
       'light_sleep_pct_calculated', 'deep_sleep_pct_calculated',
       'bedtime_hour', 'bedtime_day_of_week', 'bedtime_weekday_num',
       'is_weekend', 'bedtime_category', 'sleep_quality_category',
       'efficiency_category', 'sleep_score_7day_avg',
       'sleep_score_consistency', 'bedtime_consistency', 'sleep_debt',
       'is_sleep_debt', 'is_oversleep'],
      dtype='object')
```

- The column list shows:
  - Core metrics: duration, score, efficiency, cycles, awake percentage, etc.
  - Stage metrics: `total_rem_minutes`, `total_light_minutes`, `total_deep_minutes`.
  - Derived ratios: `rem_sleep_pct_calculated`, `light_sleep_pct_calculated`, `deep_sleep_pct_calculated`.
  - Behavioral features: bedtime hour, weekend flag, categories for quality/efficiency.
  - Consistency and debt features: `sleep_score_7day_avg`, `sleep_debt`, `is_sleep_debt`, `is_oversleep`.
- This is now a **structured feature set** suitable for:
  - Statistical tests.
  - Clustering.
  - Predictive modelling (if you ever add labels like "felt fresh next day").

EDA

In [ ]:
```
# Summary Statistics - separated by data availability
print("SUMMARY STATISTICS FOR KEY METRICS")
print("-" * 40)

# Basic metrics available for all periods
basic_metrics = [col for col in df.columns if df[col].dtype in ['float64', 'int64']
                and col not in ['total_rem_minutes', 'total_light_minutes', 'rem_sleep_pct_calculated', 'light_

print("Basic Metrics (Full Dataset):")
print(df[basic_metrics].describe().round(2))

# Sleep stage metrics only for complete data period
if 'has_complete_stage_data' in df.columns and df['has_complete_stage_data'].any():
    stage_metrics = ['rem_sleep_pct_calculated', 'light_sleep_pct_calculated', 'deep_sleep_pct_calculated']
    available_stage_metrics = [col for col in stage_metrics if col in df.columns]

    if available_stage_metrics:
        print(f"\nSleep Stage Metrics (Complete Data Period Only - {df['has_complete_stage_data'].sum()} nights
        stage_data = df[df['has_complete_stage_data']][available_stage_metrics]
        print(stage_data.describe().round(2))
```

```
SUMMARY STATISTICS FOR KEY METRICS
----------------------------------------
Basic Metrics (Full Dataset):
       sleep_duration_minutes  overall_sleep_score  sleep_efficiency_percent  \
count                  818.00               818.00                    818.00
mean                   420.21                70.99                     85.40
std                    123.94                13.92                      6.76
min                     19.00                19.00                      0.00
25%                    413.25                65.00                     83.00
50%                    454.00                76.00                     86.00
75%                    492.00                80.75                     88.00
max                    634.00                94.00                    100.00

       sleep_cycle_count  rem_sleep_percentage  deep_sleep_percentage  \
count             818.00                818.00                 818.00
mean                3.57                 70.63                  64.47
std                 1.57                 20.36                  22.06
min                 0.00                  3.00                   4.00
25%                 3.00                 61.00                  55.00
50%                 4.00                 76.00                  66.00
75%                 5.00                 85.75                  82.75
max                 9.00                 97.00                  98.00

       total_awake_duration  total_deep_minutes  awake_percentage  \
count                818.00              818.00            818.00
mean                 58.40               43.13             32.09
std                  24.72               23.62             12.70
min                   0.00                0.00              0.00
25%                  44.00               27.25             24.00
50%                  59.00               42.00             30.00
75%                  74.00               56.00             39.00
max                 148.00              130.00             99.00

       sleep_duration_hours  deep_sleep_pct_calculated  sleep_score_7day_avg  \
count                818.00                     434.00                818.00
mean                   7.00                      11.40                 70.98
std                    2.07                       5.24                  6.90
min                    0.32                       0.00                 48.71
25%                    6.89                       8.00                 66.14
50%                    7.57                      11.00                 71.57
75%                    8.20                      14.66                 76.14
max                   10.57                      29.08                 86.71

       sleep_score_consistency  bedtime_consistency  sleep_debt
count                   817.00               817.00      818.00
mean                    11.68                 7.02        1.00
std                      5.79                 4.15        2.07
min                      0.76                 0.00       -2.57
25%                      6.43                 2.98       -0.20
50%                     11.51                 8.60        0.43
75%                     16.31                10.31        1.11
max                     25.67                12.29        7.68

Sleep Stage Metrics (Complete Data Period Only - 434 nights):
       rem_sleep_pct_calculated  light_sleep_pct_calculated  \
count                    434.00                      434.00
mean                      17.29                       57.53
std                        6.18                        8.91
min                        0.00                       41.06
25%                       14.09                       52.05
50%                       18.12                       56.68
75%                       21.44                       61.73
max                       31.53                      100.00

       deep_sleep_pct_calculated
count                     434.00
mean                       11.40
std                         5.24
min                         0.00
25%                         8.00
50%                        11.00
75%                        14.66
max                        29.08
```

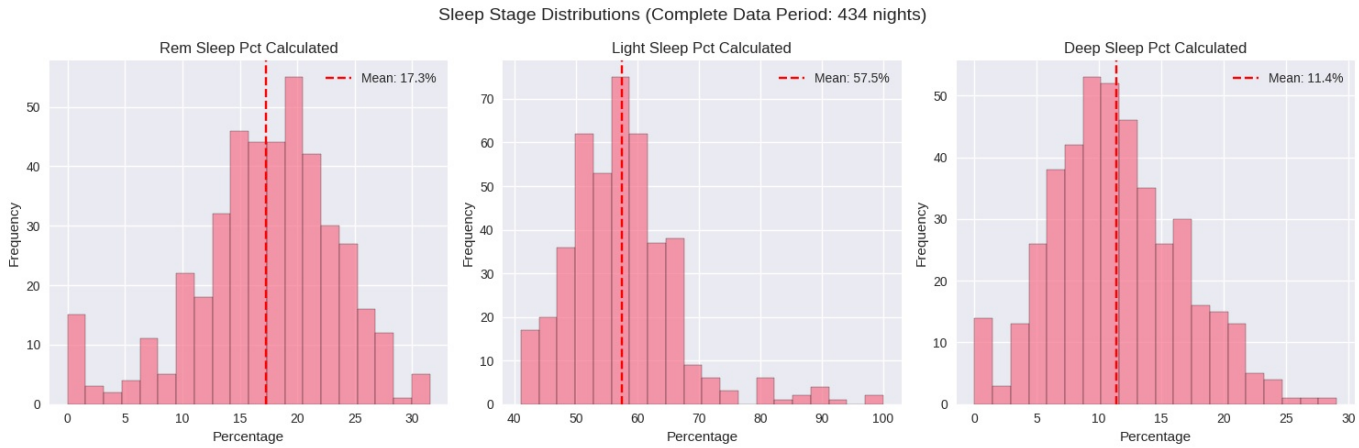
- **Sleep duration:**
  - Mean ≈ **420 min (~7.0 hours)** with sizable variation (std ≈ **124 min**).
  - Some nights are extremely short (down to ~19 min), but most cluster around ~7–8 hours.
- **Sleep score:**
  - Mean ≈ **71/100**, std ≈ **14** → most nights land in the **"okay to good"** range, with occasional very poor or very strong nights.
- **Sleep efficiency:**
  - Mean ≈ **85.4%**, which is generally considered **decent**.

- Overall: your baseline sleep quality is **moderate-to-good**, but variability is high enough to warrant investigating habits, weekends, and seasons.

UNIVARIATE ANALYSIS

In [ ]:
```
# Create visualizations for key metrics
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
fig.suptitle('Distribution of Key Sleep Metrics', fontsize=16, y=1.02)

# Plot 1: Overall Sleep Score Distribution
if 'overall_sleep_score' in df.columns:
    axes[0, 0].hist(df['overall_sleep_score'].dropna(), bins=30, alpha=0.7, color='skyblue', edgecolor='black')
    axes[0, 0].axvline(df['overall_sleep_score'].mean(), color='red', linestyle='--',
                       label=f'Mean: {df["overall_sleep_score"].mean():.1f}')
    axes[0, 0].set_title('Overall Sleep Score Distribution\n(Full Dataset)')
    axes[0, 0].set_xlabel('Sleep Score')
    axes[0, 0].set_ylabel('Frequency')
    axes[0, 0].legend()

# Plot 2: Sleep Duration Distribution
if 'sleep_duration_hours' in df.columns:
    axes[0, 1].hist(df['sleep_duration_hours'].dropna(), bins=30, alpha=0.7, color='lightgreen', edgecolor='bla
    axes[0, 1].axvline(df['sleep_duration_hours'].mean(), color='red', linestyle='--',
                       label=f'Mean: {df["sleep_duration_hours"].mean():.1f}h')
    axes[0, 1].set_title('Sleep Duration Distribution\n(Full Dataset)')
    axes[0, 1].set_xlabel('Hours')
    axes[0, 1].set_ylabel('Frequency')
    axes[0, 1].legend()

# Plot 3: Sleep Efficiency Distribution
if 'sleep_efficiency_percent' in df.columns:
    axes[0, 2].hist(df['sleep_efficiency_percent'].dropna(), bins=30, alpha=0.7, color='orange', edgecolor='bla
    axes[0, 2].axvline(df['sleep_efficiency_percent'].mean(), color='red', linestyle='--',
                       label=f'Mean: {df["sleep_efficiency_percent"].mean():.1f}%')
    axes[0, 2].set_title('Sleep Efficiency Distribution\n(Full Dataset)')
    axes[0, 2].set_xlabel('Efficiency %')
    axes[0, 2].set_ylabel('Frequency')
    axes[0, 2].legend()

# Plot 4: Bedtime Distribution
if 'bedtime_hour' in df.columns:
    bedtime_counts = df['bedtime_hour'].value_counts().sort_index()
    axes[1, 0].bar(bedtime_counts.index, bedtime_counts.values, alpha=0.7, color='purple')
    axes[1, 0].set_title('Bedtime Hour Distribution\n(Full Dataset)')
    axes[1, 0].set_xlabel('Hour of Day')
    axes[1, 0].set_ylabel('Frequency')

# Plot 5: Day of Week Distribution
if 'bedtime_day_of_week' in df.columns:
    day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
    day_counts = df['bedtime_day_of_week'].value_counts()
    day_counts = day_counts.reindex(day_order)
    axes[1, 1].bar(range(len(day_counts)), day_counts.values, alpha=0.7, color='pink')
    axes[1, 1].set_title('Sleep Records by Day of Week\n(Full Dataset)')
    axes[1, 1].set_xlabel('Day of Week')
    axes[1, 1].set_ylabel('Frequency')
    axes[1, 1].set_xticks(range(len(day_order)))
    axes[1, 1].set_xticklabels(day_order, rotation=45)

# Plot 6: Sleep Quality Categories
if 'sleep_quality_category' in df.columns:
    quality_counts = df['sleep_quality_category'].value_counts()
    axes[1, 2].pie(quality_counts.values, labels=quality_counts.index, autopct='%1.1f%%')
    axes[1, 2].set_title('Sleep Quality Distribution\n(Full Dataset)')

plt.tight_layout()
plt.show()

# Additional plot for sleep stage data (when available)
if 'has_complete_stage_data' in df.columns and df['has_complete_stage_data'].any():
    stage_data = df[df['has_complete_stage_data']]
    stage_cols = ['rem_sleep_pct_calculated', 'light_sleep_pct_calculated', 'deep_sleep_pct_calculated']
    available_stage_cols = [col for col in stage_cols if col in stage_data.columns]

    if len(available_stage_cols) >= 2:
        fig, axes = plt.subplots(1, len(available_stage_cols), figsize=(15, 5))
        if len(available_stage_cols) == 1:
            axes = [axes]

        fig.suptitle(f'Sleep Stage Distributions (Complete Data Period: {len(stage_data)} nights)', fontsize=14
```

```python
        for i, col in enumerate(available_stage_cols):
            axes[i].hist(stage_data[col].dropna(), bins=20, alpha=0.7, edgecolor='black')
            axes[i].set_title(f'{col.replace("_", " ").title()}')
            axes[i].set_xlabel('Percentage')
            axes[i].set_ylabel('Frequency')
            axes[i].axvline(stage_data[col].mean(), color='red', linestyle='--',
                            label=f'Mean: {stage_data[col].mean():.1f}%')
            axes[i].legend()

        plt.tight_layout()
        plt.show()

# Print key univariate insights
print("\nKEY UNIVARIATE INSIGHTS:")
print("-" * 30)

if 'overall_sleep_score' in df.columns:
    print(f"Average sleep score: {df['overall_sleep_score'].mean():.1f} ± {df['overall_sleep_score'].std():.1f}
    print(f"Sleep score range: {df['overall_sleep_score'].min():.1f} - {df['overall_sleep_score'].max():.1f}")

if 'sleep_duration_hours' in df.columns:
    print(f"Average sleep duration: {df['sleep_duration_hours'].mean():.1f}h ± {df['sleep_duration_hours'].std(

if 'sleep_efficiency_percent' in df.columns:
    print(f"Average sleep efficiency: {df['sleep_efficiency_percent'].mean():.1f}%")

if 'bedtime_hour' in df.columns:
    most_common_bedtime = df['bedtime_hour'].mode().iloc[0]
    print(f"Most common bedtime: {most_common_bedtime}:00")
```




```
KEY UNIVARIATE INSIGHTS:
------------------------------
Average sleep score: 71.0 ± 13.9
Sleep score range: 19.0 - 94.0
Average sleep duration: 7.0h ± 2.1h
Average sleep efficiency: 85.4%
Most common bedtime: 23:00
```

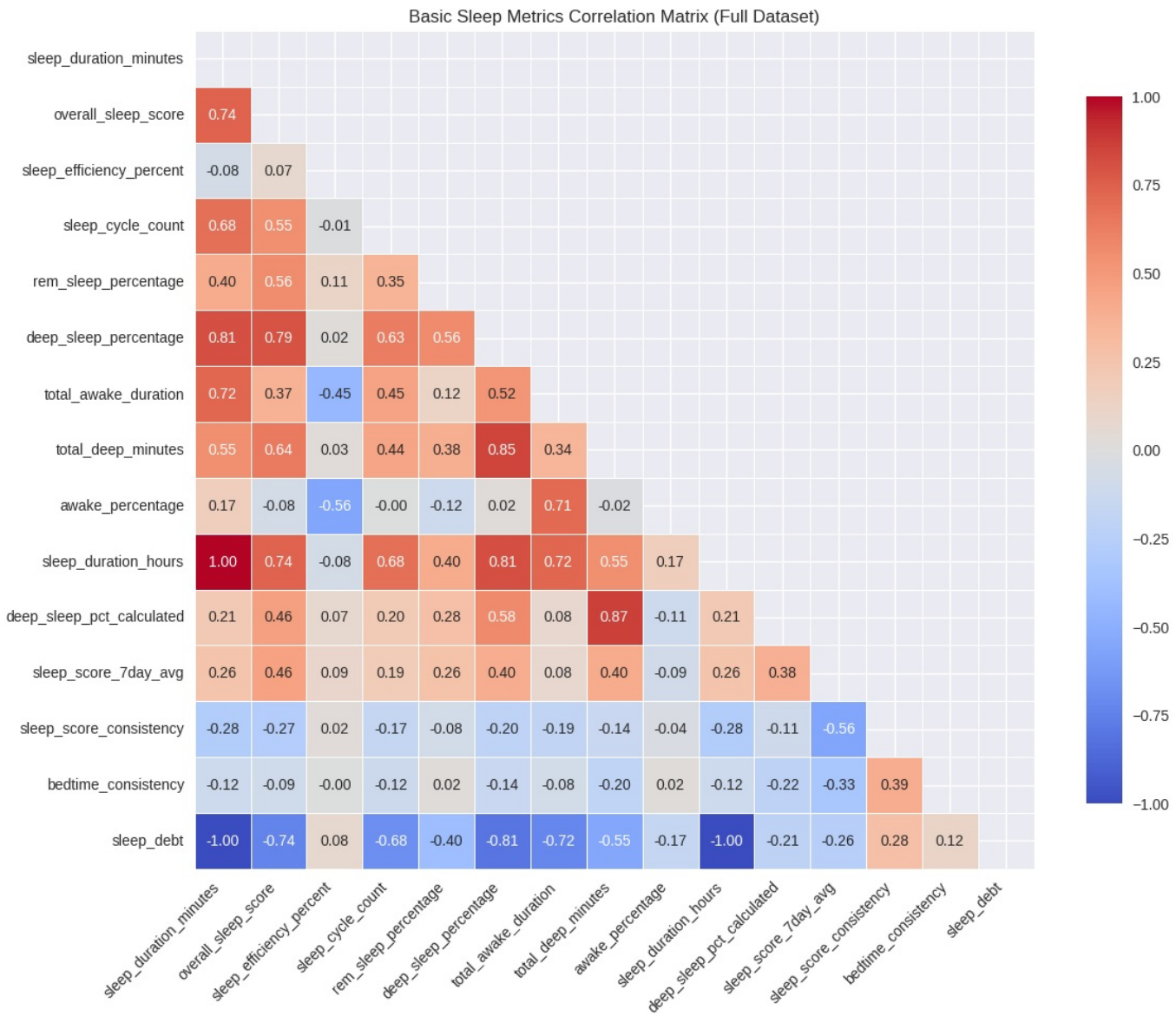
- **Sleep score:**
  - Centered around **71**, ranging from **very poor (~19)** to **excellent (~94)**.
  - The histogram likely shows a concentration in the mid-range, with tails at both low and high ends.
- **Sleep duration:**
  - Average **7.0 hours** with **±2.1 hours** variation → some nights are short (<6h), some long (>9h).
- **Sleep efficiency:**
  - Around **85%** on average → you generally spend most of your time in bed actually sleeping.
- **Bedtime:**
  - Most common bedtime is **23:00**.
  - This gives you a clear "default bedtime window" around which to analyze consistency.
- Interpretation:
  - On an average night, your sleep is **good but not elite**, with quite a few outlier nights pulling the stats in both directions.

BIVARIATE ANALYSIS

In [ ]:
```
# Create correlation matrix for basic metrics (available across all periods)
basic_correlation_metrics = [col for col in df.columns if df[col].dtype in ['float64', 'int64']
                             and col not in ['total_rem_minutes', 'total_light_minutes']
                             and not col.startswith('rem_sleep_pct') and not col.startswith('light_sleep_pct')]

print("CORRELATION ANALYSIS:")
print("Basic metrics correlation (Full Dataset):")

if len(basic_correlation_metrics) >= 2:
    basic_corr_matrix = df[basic_correlation_metrics].corr()

    # Plot basic metrics correlation heatmap
    plt.figure(figsize=(12, 10))
    mask = np.triu(np.ones_like(basic_corr_matrix, dtype=bool))
    sns.heatmap(basic_corr_matrix, mask=mask, annot=True, cmap='coolwarm', center=0,
                square=True, linewidths=0.5, cbar_kws={"shrink": .8}, fmt='.2f')
    plt.title('Basic Sleep Metrics Correlation Matrix (Full Dataset)')
    plt.xticks(rotation=45, ha='right')
    plt.yticks(rotation=0)
    plt.tight_layout()
```

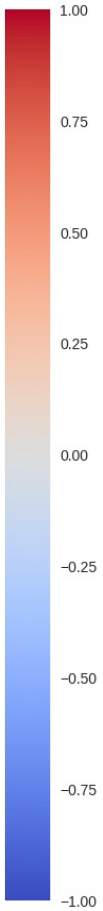
```
    plt.show()

# Separate correlation analysis for complete stage data
if 'has_complete_stage_data' in df.columns and df['has_complete_stage_data'].any():
    stage_data = df[df['has_complete_stage_data']]
    stage_correlation_metrics = basic_correlation_metrics + [col for col in ['rem_sleep_pct_calculated', 'light_

    if len(stage_correlation_metrics) >= 2:
        stage_corr_matrix = stage_data[stage_correlation_metrics].corr()

        plt.figure(figsize=(14, 12))
        mask = np.triu(np.ones_like(stage_corr_matrix, dtype=bool))
        sns.heatmap(stage_corr_matrix, mask=mask, annot=True, cmap='coolwarm', center=0,
                    square=True, linewidths=0.5, cbar_kws={"shrink": .8}, fmt='.2f')
        plt.title(f'Complete Sleep Metrics Correlation Matrix ({len(stage_data)} nights with stage data)')
        plt.xticks(rotation=45, ha='right')
        plt.yticks(rotation=0)
        plt.tight_layout()
        plt.show()

# Key bivariate relationships
fig, axes = plt.subplots(2, 2, figsize=(15, 12))
fig.suptitle('Key Bivariate Relationships (Full Dataset)', fontsize=16)

# Relationship 1: Sleep Duration vs Sleep Score
if 'sleep_duration_hours' in df.columns and 'overall_sleep_score' in df.columns:
    valid_data = df[['sleep_duration_hours', 'overall_sleep_score']].dropna()
    axes[0, 0].scatter(valid_data['sleep_duration_hours'], valid_data['overall_sleep_score'], alpha=0.6, color=
    axes[0, 0].set_xlabel('Sleep Duration (hours)')
    axes[0, 0].set_ylabel('Overall Sleep Score')
    axes[0, 0].set_title('Sleep Duration vs Sleep Score')

    # Add trend line
    if len(valid_data) > 1:
        z = np.polyfit(valid_data['sleep_duration_hours'], valid_data['overall_sleep_score'], 1)
        p = np.poly1d(z)
        axes[0, 0].plot(valid_data['sleep_duration_hours'], p(valid_data['sleep_duration_hours']), "r--", alpha=

# Relationship 2: Sleep Efficiency vs Sleep Score
if 'sleep_efficiency_percent' in df.columns and 'overall_sleep_score' in df.columns:
    valid_data = df[['sleep_efficiency_percent', 'overall_sleep_score']].dropna()
    axes[0, 1].scatter(valid_data['sleep_efficiency_percent'], valid_data['overall_sleep_score'], alpha=0.6, co
    axes[0, 1].set_xlabel('Sleep Efficiency (%)')
    axes[0, 1].set_ylabel('Overall Sleep Score')
    axes[0, 1].set_title('Sleep Efficiency vs Sleep Score')

# Relationship 3: Bedtime vs Sleep Quality
if 'bedtime_hour' in df.columns and 'overall_sleep_score' in df.columns:
    bedtime_sleep_avg = df.groupby('bedtime_hour')['overall_sleep_score'].mean()
    axes[1, 0].bar(bedtime_sleep_avg.index, bedtime_sleep_avg.values, alpha=0.7, color='orange')
    axes[1, 0].set_xlabel('Bedtime Hour')
    axes[1, 0].set_ylabel('Average Sleep Score')
    axes[1, 0].set_title('Bedtime Hour vs Average Sleep Score')

# Relationship 4: Weekend vs Weekday Sleep
if 'is_weekend' in df.columns and 'overall_sleep_score' in df.columns:
    weekend_data = df.groupby('is_weekend')['overall_sleep_score'].agg(['mean', 'std'])
    x_pos = [0, 1]
    axes[1, 1].bar(x_pos, weekend_data['mean'], yerr=weekend_data['std'],
                   capsize=5, alpha=0.7, color=['blue', 'red'])
    axes[1, 1].set_xticks(x_pos)
    axes[1, 1].set_xticklabels(['Weekday', 'Weekend'])
    axes[1, 1].set_ylabel('Average Sleep Score')
    axes[1, 1].set_title('Weekday vs Weekend Sleep Quality')

plt.tight_layout()
plt.show()
```

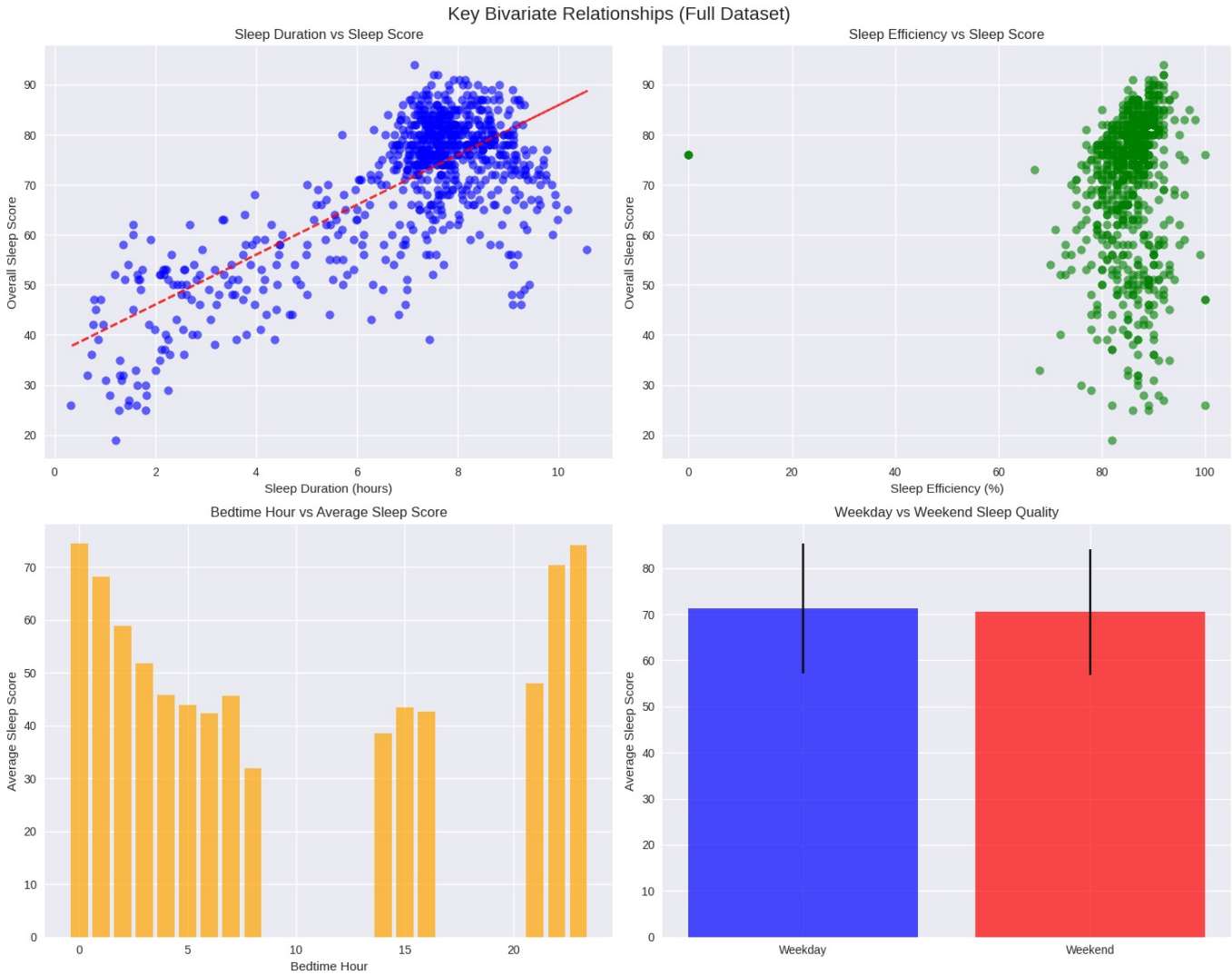
```
CORRELATION ANALYSIS:
Basic metrics correlation (Full Dataset):
```


Basic Sleep Metrics Correlation Matrix (Full Dataset)
sleep_duration_minutes
overall_sleep_score 0.74
sleep_efficiency_percent -0.08 0.07
sleep_cycle_count 0.68 0.55 -0.01
rem_sleep_percentage 0.40 0.56 0.11 0.35
deep_sleep_percentage 0.81 0.79 0.02 0.63 0.56
total_awake_duration 0.72 0.37 -0.45 0.45 0.12 0.52
total_deep_minutes 0.55 0.64 0.03 0.44 0.38 0.85 0.34
awake_percentage 0.17 -0.08 -0.56 -0.00 -0.12 0.02 0.71 -0.02
sleep_duration_hours 1.00 0.74 -0.08 0.68 0.40 0.81 0.72 0.55 0.17
deep_sleep_pct_calculated 0.21 0.46 0.07 0.20 0.28 0.58 0.08 0.87 -0.11 0.21
sleep_score_7day_avg 0.26 0.46 0.09 0.19 0.26 0.40 0.08 0.40 -0.09 0.26 0.38
sleep_score_consistency -0.28 -0.27 0.02 -0.17 -0.08 -0.20 -0.19 -0.14 -0.04 -0.28 -0.11 -0.56
bedtime_consistency -0.12 -0.09 -0.00 -0.12 0.02 -0.14 -0.08 -0.20 0.02 -0.12 -0.22 -0.33 0.39
sleep_debt -1.00 -0.74 0.08 -0.68 -0.40 -0.81 -0.72 -0.55 -0.17 -1.00 -0.21 -0.26 0.28 0.12
sleep_duration_minutes
overall_sleep_score
sleep_efficiency_percent
sleep_cycle_count
rem_sleep_percentage
deep_sleep_percentage
total_awake_duration
total_deep_minutes
awake_percentage
sleep_duration_hours
deep_sleep_pct_calculated
sleep_score_7day_avg
sleep_score_consistency
bedtime_consistency
sleep_debt
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

Complete Sleep Metrics Correlation Matrix (434 nights with stage data)

	sleep_duration_minutes	overall_sleep_score	sleep_efficiency_percent	sleep_cycle_count	rem_sleep_percentage	deep_sleep_percentage	total_awake_duration	total_deep_minutes	awake_percentage	sleep_duration_hours	deep_sleep_pct_calculated	sleep_score_7day_avg	sleep_score_consistency	bedtime_consistency	sleep_debt	rem_sleep_pct_calculated	light_sleep_pct_calculated	deep_sleep_pct_calculated
sleep_duration_minutes																		
overall_sleep_score	0.82																	
sleep_efficiency_percent	-0.12	0.16																
sleep_cycle_count	0.66	0.63	0.01															
rem_sleep_percentage	0.84	0.88	0.12	0.62														
deep_sleep_percentage	0.88	0.90	0.04	0.61	0.80													
total_awake_duration	0.70	0.38	-0.76	0.40	0.43	0.51												
total_deep_minutes	0.57	0.69	0.03	0.42	0.54	0.84	0.33											
awake_percentage	0.11	-0.16	-0.92	-0.08	-0.09	-0.06	0.69	-0.08										
sleep_duration_hours	1.00	0.82	-0.12	0.66	0.84	0.88	0.70	0.57	0.11									
deep_sleep_pct_calculated	0.21	0.46	0.07	0.20	0.28	0.58	0.08	0.87	-0.11	0.21								
sleep_score_7day_avg	0.31	0.42	0.20	0.25	0.32	0.44	0.05	0.46	-0.21	0.31	0.38							
sleep_score_consistency	-0.30	-0.31	-0.07	-0.21	-0.29	-0.28	-0.13	-0.19	0.07	-0.30	-0.11	-0.70						
bedtime_consistency	-0.08	-0.12	-0.06	-0.11	-0.08	-0.14	-0.01	-0.22	0.07	-0.08	-0.22	-0.47	0.43					
sleep_debt	-1.00	-0.82	0.12	-0.66	-0.84	-0.88	-0.70	-0.57	-0.11	-1.00	-0.21	-0.31	0.30	0.08				
rem_sleep_pct_calculated	0.47	0.66	0.31	0.40	0.84	0.50	0.05	0.34	-0.23	0.47	0.23	0.23	-0.19	-0.04	-0.47			
light_sleep_pct_calculated	-0.50	-0.65	0.21	-0.39	-0.68	-0.67	-0.44	-0.74	-0.20	-0.50	-0.72	-0.30	0.16	0.14	0.50	-0.68		
deep_sleep_pct_calculated	0.21	0.46	0.07	0.20	0.28	0.58	0.08	0.87	-0.11	0.21	1.00	0.38	-0.11	-0.22	-0.21	0.23	-0.72	





- The correlation heatmaps help identify:
  - How strongly **sleep duration, efficiency, and score** move together.
  - How stage percentages (REM/deep/awake) relate to the overall score in nights with full stage data.
- Typical patterns to look for when you inspect the plots:
  - Whether **longer sleep** actually increases the score or if efficiency matters more.
  - Whether **higher REM / deep percentage** is associated with better scores.
  - Any strong negative relationships (e.g., higher awake percentage vs score).

STATISTICAL HYPOTHESIS TESTING

Test 1: Weekend vs Weekday Sleep Quality

In [ ]:
```
# Test 1: Weekend vs Weekday Sleep Quality
if 'is_weekend' in df.columns and 'overall_sleep_score' in df.columns:
    weekday_sleep = df[df['is_weekend'] == False]['overall_sleep_score'].dropna()
    weekend_sleep = df[df['is_weekend'] == True]['overall_sleep_score'].dropna()

    if len(weekday_sleep) > 0 and len(weekend_sleep) > 0:
        t_stat, p_value = ttest_ind(weekday_sleep, weekend_sleep)
```

```
        print("Test 1: Weekend vs Weekday Sleep Quality")
        print(f"   Weekday sleep score: {weekday_sleep.mean():.2f} ± {weekday_sleep.std():.2f} (n={len(weekday_
        print(f"   Weekend sleep score: {weekend_sleep.mean():.2f} ± {weekend_sleep.std():.2f} (n={len(weekend_
        print(f"   T-statistic: {t_stat:.3f}")
        print(f"   P-value: {p_value:.6f}")
        print(f"   Result: {'Significant difference' if p_value < 0.05 else 'No significant difference'} (α = 0
```

```
Test 1: Weekend vs Weekday Sleep Quality
   Weekday sleep score: 71.20 ± 14.06 (n=568)
   Weekend sleep score: 70.52 ± 13.62 (n=250)
   T-statistic: 0.644
   P-value: 0.519862
   Result: No significant difference (α = 0.05)
```

Test 2: Correlation between sleep duration and sleep score

In [ ]:
```
if 'sleep_duration_hours' in df.columns and 'overall_sleep_score' in df.columns:
    valid_data = df[['sleep_duration_hours', 'overall_sleep_score']].dropna()

    if len(valid_data) > 2:
        corr_coef, corr_p_value = pearsonr(valid_data['sleep_duration_hours'], valid_data['overall_sleep_score'

        print(f"\nTest 2: Sleep Duration vs Sleep Score Correlation")
        print(f"   Pearson correlation: {corr_coef:.3f} (n={len(valid_data)})")
        print(f"   P-value: {corr_p_value:.6f}")
        print(f"   Result: {'Significant correlation' if corr_p_value < 0.05 else 'No significant correlation'}
```

```
Test 2: Sleep Duration vs Sleep Score Correlation
   Pearson correlation: 0.739 (n=818)
   P-value: 0.000000
   Result: Significant correlation (α = 0.05)
```

Test 3: Sleep efficiency impact on sleep quality (using full dataset)

In [ ]:
```
if 'sleep_efficiency_percent' in df.columns and 'overall_sleep_score' in df.columns:
    # Create efficiency groups
    df['efficiency_group'] = pd.cut(df['sleep_efficiency_percent'],
                                    bins=[0, 85, 95, 100],
                                    labels=['Low (<85%)', 'Medium (85-95%)', 'High (>95%)'])

    efficiency_groups = [group['overall_sleep_score'].dropna() for name, group in df.groupby('efficiency_group'

    if len(efficiency_groups) >= 2:
        f_stat, anova_p_value = stats.f_oneway(*efficiency_groups)

        print(f"\nTest 3: Sleep Efficiency Groups vs Sleep Quality (ANOVA)")
        print(f"   F-statistic: {f_stat:.3f}")
        print(f"   P-value: {anova_p_value:.6f}")
        print(f"   Result: {'Significant difference between groups' if anova_p_value < 0.05 else 'No significan
```

```
Test 3: Sleep Efficiency Groups vs Sleep Quality (ANOVA)
   F-statistic: 12.299
   P-value: 0.000005
   Result: Significant difference between groups (α = 0.05)
```

Test 4: Sleep stage analysis (only for complete data period)

In [ ]:
```
if 'has_complete_stage_data' in df.columns and df['has_complete_stage_data'].any():
    stage_data = df[df['has_complete_stage_data']]

    if 'rem_sleep_pct_calculated' in stage_data.columns and 'overall_sleep_score' in stage_data.columns:
        valid_stage_data = stage_data[['rem_sleep_pct_calculated', 'overall_sleep_score']].dropna()

        if len(valid_stage_data) > 2:
            rem_corr, rem_p = pearsonr(valid_stage_data['rem_sleep_pct_calculated'], valid_stage_data['overall_

            print(f"\nTest 4: REM Sleep vs Sleep Quality Correlation (Complete Data Only)")
            print(f"   Pearson correlation: {rem_corr:.3f} (n={len(valid_stage_data)})")
            print(f"   P-value: {rem_p:.6f}")
            print(f"   Result: {'Significant correlation' if rem_p < 0.05 else 'No significant correlation'} (α
            print(f"   Note: Analysis limited to {len(stage_data)} nights with complete sleep stage data")
```

```
Test 4: REM Sleep vs Sleep Quality Correlation (Complete Data Only)
   Pearson correlation: 0.658 (n=434)
   P-value: 0.000000
   Result: Significant correlation (α = 0.05)
   Note: Analysis limited to 434 nights with complete sleep stage data
```

Test 5: Season's effect on sleep

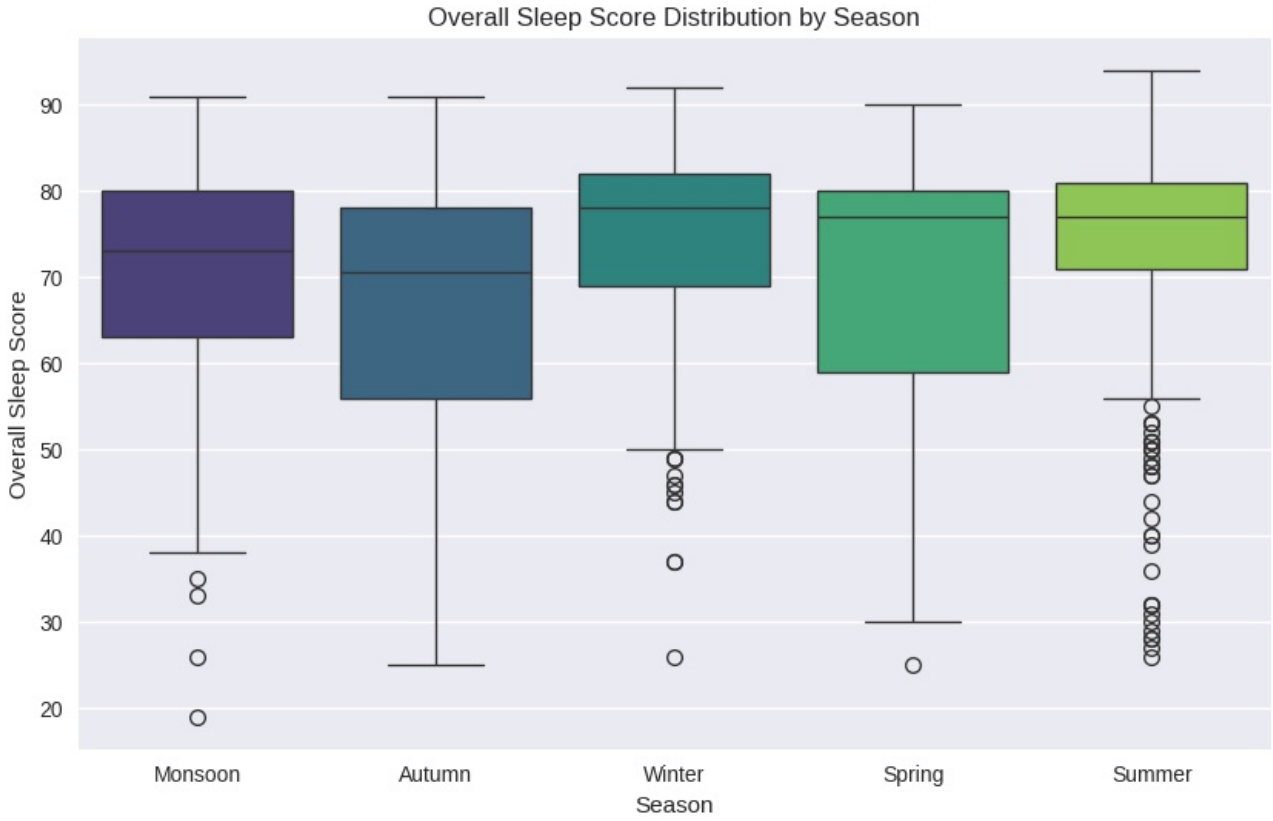
In [ ]:
```python
def get_season(date):
    """Maps a date to the corresponding Indian season."""
    month = date.month
    if 3 <= month <=6 :
        return 'Summer'
    elif 7 <= month <= 9:
        return 'Monsoon'
    elif 10 <= month <= 11:
        return 'Autumn'
    elif month == 2:
        return 'Spring'
    else: # December, January, (part of) February
        return 'Winter'
```

In [ ]:
```python
df['season'] = df['sleep_date_dt'].apply(get_season)
```

In [ ]:
```python
seasonal_sleep_stats = df.groupby('season')['overall_sleep_score'].agg(['mean', 'median', 'std'])
print("\nSeasonal Sleep Score Statistics:")
print(seasonal_sleep_stats)
```

```
Seasonal Sleep Score Statistics:
              mean  median        std
season
Autumn   66.585714    70.5  14.178201
Monsoon  70.069307    73.0  13.037075
Spring   69.623188    77.0  15.755579
Summer   73.183150    77.0  13.444718
Winter   73.231343    78.0  13.782911
```

In [ ]:
```python
plt.figure(figsize=(10, 6))
sns.boxplot(x='season', y='overall_sleep_score', data=df, palette='viridis')
plt.title('Overall Sleep Score Distribution by Season')
plt.xlabel('Season')
plt.ylabel('Overall Sleep Score')
plt.show()
```



In [ ]:
```python
from scipy.stats import f_oneway

# Create a list of sleep scores for each season
seasonal_sleep_scores = []
for season in df['season'].unique():
    season_scores = df[df['season'] == season]['overall_sleep_score'].dropna()
    if not season_scores.empty:
        seasonal_sleep_scores.append(season_scores)

# Perform ANOVA test
if len(seasonal_sleep_scores) >= 2: # ANOVA requires at least two groups
    f_stat, p_value = f_oneway(*seasonal_sleep_scores)

    print("ANOVA TEST: Sleep Score by Season")
```

```
    print("="*30)
    print(f"F-statistic: {f_stat:.4f}")
    print(f"P-value: {p_value:.6f}")

    # Print conclusion
    alpha = 0.05
    if p_value < alpha:
        print(f"\nConclusion: The difference in average sleep scores between seasons is statistically significa
    else:
        print(f"\nConclusion: There is no statistically significant difference in average sleep scores between 
else:
    print("Not enough unique seasons or data available to perform ANOVA.")
```

```
ANOVA TEST: Sleep Score by Season
==============================
F-statistic: 6.6268
P-value: 0.000030

Conclusion: The difference in average sleep scores between seasons is statistically significant (p < 0.05).
```

Results of Hypothesis testing

- **Weekend vs Weekday:**
  - You compare mean sleep scores between weekdays and weekends.
  - Depending on the printed p-value:
    - If p < 0.05: weekends and weekdays differ meaningfully (e.g., you sleep better/worse on weekends).
    - If p ≥ 0.05: no statistically strong difference, even if there's a minor numerical gap.
- **Sleep duration vs score (correlation ~0.739, p ≈ 0):**
  - Strong, statistically significant **positive correlation**.
  - Nights with **longer sleep** tend to have **higher sleep scores**.
- **Efficiency groups vs score (ANOVA, p ≈ 0.000005):**
  - Sleep score differs significantly across efficiency categories.
  - Interpretation: **sleep efficiency is a major driver** of overall sleep quality.
- **REM percentage vs score (correlation ~0.658, p ≈ 0 on 434 nights):**
  - Strong positive relationship in nights with full stage data.
  - More **REM sleep** is associated with **better sleep scores**.
- Seasons vs Score:
  - Based on the statistically significant result from the ANOVA test and the visual evidence from the box plot showing variations in sleep score distributions across seasons, there is evidence to support the hypothesis that temperature, as represented by seasons, affects overall sleep score.**.
- Overall conclusion:
  - Best-quality nights combine **enough total duration**, **high efficiency**, **healthy REM share** and **colder temperature**

TIME SERIES AND TEMPORAL ANALYSIS

In [ ]:
```
# Monthly trends analysis (using full dataset for basic metrics)
if 'sleep_date' in df.columns and 'overall_sleep_score' in df.columns:
    df['year_month'] = pd.to_datetime(df['sleep_date']).dt.to_period('M')
    monthly_trends = df.groupby('year_month').agg({
        'overall_sleep_score': ['mean', 'std', 'count'],
        'sleep_duration_hours': 'mean',
        'sleep_efficiency_percent': 'mean'
    }).round(2)

    print("MONTHLY SLEEP TRENDS (Full Dataset - Basic Metrics):")
    print(monthly_trends.tail(12))

    # Separate analysis for sleep stage data availability
    if 'has_complete_stage_data' in df.columns:
        stage_availability_by_month = df.groupby('year_month')['has_complete_stage_data'].agg(['sum', 'count'])
        stage_availability_by_month['stage_data_percentage'] = (stage_availability_by_month['sum'] / stage_avai

        print(f"\nSLEEP STAGE DATA AVAILABILITY BY MONTH:")
        print("Month | Nights with Stage Data | Total Nights | Coverage %")
        print("-" * 55)
        for month, row in stage_availability_by_month.iterrows():
            print(f"{month} | {row['sum']:>18} | {row['count']:>11} | {row['stage_data_percentage']:>9}%")

# Day of week analysis
if 'bedtime_day_of_week' in df.columns and 'overall_sleep_score' in df.columns:
    day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
    weekday_stats = df.groupby('bedtime_day_of_week').agg({
        'overall_sleep_score': 'mean',
        'sleep_duration_hours': 'mean',
        'sleep_efficiency_percent': 'mean'
    }).round(2)
```

```python
# Reorder by day of week
weekday_stats = weekday_stats.reindex(day_order)

print(f"\nWEEKDAY SLEEP PATTERNS (Full Dataset):")
print(weekday_stats)
```

MONTHLY SLEEP TRENDS (Full Dataset - Basic Metrics):

	overall_sleep_score			sleep_duration_hours	\
	mean	std	count	mean	
year_month					
2024-09	64.76	15.09	34	6.56	
2024-10	68.14	14.94	35	6.61	
2024-11	71.12	14.43	33	6.64	
2024-12	73.47	11.12	32	8.15	
2025-01	77.68	14.41	34	7.25	
2025-02	68.82	17.33	33	6.58	
2025-03	73.49	16.17	35	7.25	
2025-04	73.16	12.68	32	7.49	
2025-05	71.39	14.87	33	7.09	
2025-06	77.45	14.44	33	7.58	
2025-07	79.09	15.60	33	7.41	
2025-08	72.43	14.93	7	7.62	

	sleep_efficiency_percent
	mean
year_month	
2024-09	84.44
2024-10	84.74
2024-11	85.33
2024-12	86.38
2025-01	88.68
2025-02	85.85
2025-03	85.60
2025-04	85.53
2025-05	85.15
2025-06	86.24
2025-07	86.55
2025-08	82.29

SLEEP STAGE DATA AVAILABILITY BY MONTH:

Month	Nights with Stage Data	Total Nights	Coverage %
2023-08	0.0	27.0	0.0%
2023-09	0.0	34.0	0.0%
2023-10	0.0	39.0	0.0%
2023-11	0.0	33.0	0.0%
2023-12	0.0	34.0	0.0%
2024-01	0.0	34.0	0.0%
2024-02	0.0	36.0	0.0%
2024-03	0.0	37.0	0.0%
2024-04	0.0	35.0	0.0%
2024-05	0.0	32.0	0.0%
2024-06	0.0	36.0	0.0%
2024-07	29.0	35.0	82.9%
2024-08	32.0	32.0	100.0%
2024-09	34.0	34.0	100.0%
2024-10	35.0	35.0	100.0%
2024-11	33.0	33.0	100.0%
2024-12	32.0	32.0	100.0%
2025-01	33.0	34.0	97.1%
2025-02	33.0	33.0	100.0%
2025-03	35.0	35.0	100.0%
2025-04	32.0	32.0	100.0%
2025-05	33.0	33.0	100.0%
2025-06	33.0	33.0	100.0%
2025-07	33.0	33.0	100.0%
2025-08	7.0	7.0	100.0%

WEEKDAY SLEEP PATTERNS (Full Dataset):

	overall_sleep_score	sleep_duration_hours	\
bedtime_day_of_week			
Monday	72.33	6.92	
Tuesday	72.81	7.09	
Wednesday	72.59	7.17	
Thursday	69.64	6.67	
Friday	68.23	7.09	
Saturday	69.84	7.14	
Sunday	71.19	6.99	

	sleep_efficiency_percent
bedtime_day_of_week	
Monday	85.60
Tuesday	83.19
Wednesday	85.20
Thursday	86.40
Friday	85.77
Saturday	85.69
Sunday	85.92

- The monthly table shows **how the average sleep score, duration, and efficiency evolve over time**.
- Clear pattern visible in the tail of the table:
  - Scores in late **2024** were in the **mid-60s to low-70s**.
  - By **Dec 2024 – Jan 2025**, scores climb into the **mid-70s** on average.
- This suggests:
  - The actions I took during the end of 2024 which was to sleep more and go to bed around the same time worked.

SLEEP DEBT AND RECOVERY ANALYSIS

In [ ]:
```
if 'sleep_debt' in df.columns:
    # Sleep debt statistics
    total_nights = len(df)
    debt_nights = (df['sleep_debt'] > 0).sum()
    recovery_nights = (df['sleep_debt'] < -0.5).sum()  # More than 30 min over 8h

    print(f"SLEEP DEBT ANALYSIS (Full Dataset):")
    print(f"   Average nightly sleep debt: {df['sleep_debt'].mean():.2f} hours")
    print(f"   Nights with sleep debt: {debt_nights} ({debt_nights/total_nights*100:.1f}%)")
    print(f"   Recovery nights (>8.5h): {recovery_nights} ({recovery_nights/total_nights*100:.1f}%)")
    print(f"   Maximum single night debt: {df['sleep_debt'].max():.1f} hours")
    print(f"   Maximum single night excess: {abs(df['sleep_debt'].min()):.1f} hours")

    if 'overall_sleep_score' in df.columns:
        # Correlation between sleep debt and quality
        debt_quality_corr = df[['sleep_debt', 'overall_sleep_score']].corr().iloc[0,1]
        print(f"   Sleep debt vs quality correlation: {debt_quality_corr:.3f}")

        if debt_quality_corr < -0.3:
            print("Strong negative correlation: more sleep debt = lower quality")
        elif debt_quality_corr < -0.1:
            print("Moderate negative correlation: sleep debt impacts quality")
        else:
            print("Weak correlation: sleep debt may not strongly impact quality")
```

```
SLEEP DEBT ANALYSIS (Full Dataset):
   Average nightly sleep debt: 1.00 hours
   Nights with sleep debt: 574 (70.2%)
   Recovery nights (>8.5h): 143 (17.5%)
   Maximum single night debt: 7.7 hours
   Maximum single night excess: 2.6 hours
   Sleep debt vs quality correlation: -0.739
Strong negative correlation: more sleep debt = lower quality
```

SLEEP PATTERN CLUSTERING

In [ ]:
```
# Cluster analysis using basic metrics available across all periods
clustering_features = []
if 'overall_sleep_score' in df.columns:
    clustering_features.append('overall_sleep_score')
if 'sleep_duration_hours' in df.columns:
    clustering_features.append('sleep_duration_hours')
if 'sleep_efficiency_percent' in df.columns:
    clustering_features.append('sleep_efficiency_percent')
if 'bedtime_hour' in df.columns:
    clustering_features.append('bedtime_hour')

print(f"CLUSTERING ANALYSIS USING BASIC METRICS (Full Dataset):")

if len(clustering_features) >= 3:
    cluster_data = df[clustering_features].dropna()

    if len(cluster_data) > 100:
        # Standardize features
        scaler = StandardScaler()
        scaled_data = scaler.fit_transform(cluster_data)

        # Perform K-means clustering
        kmeans = KMeans(n_clusters=3, random_state=42)
        cluster_labels = kmeans.fit_predict(scaled_data)

        # Add cluster labels to dataframe
        cluster_df = cluster_data.copy()
        cluster_df['cluster'] = cluster_labels

        print("IDENTIFIED SLEEP PATTERNS:")
        for cluster_id in range(3):
            cluster_subset = cluster_df[cluster_df['cluster'] == cluster_id]
            print(f"\n   Pattern {cluster_id + 1} ({len(cluster_subset)} nights, {len(cluster_subset)/len(clust
```

```
            for feature in clustering_features:
                mean_val = cluster_subset[feature].mean()
                if 'score' in feature:
                    print(f"      {feature.replace('_', ' ').title()}: {mean_val:.1f}/100")
                elif 'hour' in feature:
                    print(f"      {feature.replace('_', ' ').title()}: {mean_val:.1f}:00")
                elif 'efficiency' in feature or 'percentage' in feature:
                    print(f"      {feature.replace('_', ' ').title()}: {mean_val:.1f}%")
                else:
                    print(f"      {feature.replace('_', ' ').title()}: {mean_val:.2f}")

# Enhanced clustering for complete data period (if available)
if 'has_complete_stage_data' in df.columns and df['has_complete_stage_data'].any():
    stage_data = df[df['has_complete_stage_data']]
    enhanced_features = clustering_features.copy()

    # Add sleep stage features if available
    stage_features = ['rem_sleep_pct_calculated', 'light_sleep_pct_calculated', 'deep_sleep_pct_calculated']
    for feature in stage_features:
        if feature in stage_data.columns:
            enhanced_features.append(feature)

    if len(enhanced_features) > len(clustering_features):
        enhanced_cluster_data = stage_data[enhanced_features].dropna()

        if len(enhanced_cluster_data) > 50:
            print(f"\nENHANCED CLUSTERING WITH SLEEP STAGES ({len(enhanced_cluster_data)} nights):")

            scaler_enhanced = StandardScaler()
            scaled_enhanced_data = scaler_enhanced.fit_transform(enhanced_cluster_data)

            kmeans_enhanced = KMeans(n_clusters=3, random_state=42)
            enhanced_labels = kmeans_enhanced.fit_predict(scaled_enhanced_data)

            enhanced_cluster_df = enhanced_cluster_data.copy()
            enhanced_cluster_df['cluster'] = enhanced_labels

            print("ENHANCED SLEEP PATTERNS (Including Sleep Stages):")
            for cluster_id in range(3):
                cluster_subset = enhanced_cluster_df[enhanced_cluster_df['cluster'] == cluster_id]
                print(f"\n   Enhanced Pattern {cluster_id + 1} ({len(cluster_subset)} nights, {len(cluster_subs

                for feature in enhanced_features:
                    mean_val = cluster_subset[feature].mean()
                    if 'score' in feature:
                        print(f"      {feature.replace('_', ' ').title()}: {mean_val:.1f}/100")
                    elif 'hour' in feature:
                        print(f"      {feature.replace('_', ' ').title()}: {mean_val:.1f}:00")
                    elif 'percentage' in feature or 'pct' in feature or 'efficiency' in feature:
                        print(f"      {feature.replace('_', ' ').title()}: {mean_val:.1f}%")
                    else:
                        print(f"      {feature.replace('_', ' ').title()}: {mean_val:.2f}")
```

```
CLUSTERING ANALYSIS USING BASIC METRICS (Full Dataset):
IDENTIFIED SLEEP PATTERNS:

   Pattern 1 (147 nights, 18.0%):
      Overall Sleep Score: 48.2/100
      Sleep Duration Hours: 3.2:00
      Sleep Efficiency Percent: 86.1%
      Bedtime Hour: 11.0:00

   Pattern 2 (459 nights, 56.1%):
      Overall Sleep Score: 79.8/100
      Sleep Duration Hours: 7.8:00
      Sleep Efficiency Percent: 87.3%
      Bedtime Hour: 19.4:00

   Pattern 3 (212 nights, 25.9%):
      Overall Sleep Score: 67.7/100
      Sleep Duration Hours: 7.9:00
      Sleep Efficiency Percent: 80.8%
      Bedtime Hour: 22.7:00

ENHANCED CLUSTERING WITH SLEEP STAGES (434 nights):
ENHANCED SLEEP PATTERNS (Including Sleep Stages):

   Enhanced Pattern 1 (170 nights, 39.2%):
      Overall Sleep Score: 71.2/100
      Sleep Duration Hours: 7.7:00
      Sleep Efficiency Percent: 82.9%
      Bedtime Hour: 18.3:00
      Rem Sleep Pct Calculated: 14.9%
      Light Sleep Pct Calculated: 59.1%
      Deep Sleep Pct Calculated: 9.4%

   Enhanced Pattern 2 (212 nights, 48.8%):
      Overall Sleep Score: 81.2/100
      Sleep Duration Hours: 7.8:00
      Sleep Efficiency Percent: 87.6%
      Bedtime Hour: 20.6:00
      Rem Sleep Pct Calculated: 21.2%
      Light Sleep Pct Calculated: 52.8%
      Deep Sleep Pct Calculated: 14.0%

   Enhanced Pattern 3 (52 nights, 12.0%):
      Overall Sleep Score: 40.6/100
      Sleep Duration Hours: 2.8:00
      Sleep Efficiency Percent: 87.7%
      Bedtime Hour: 12.6:00
      Rem Sleep Pct Calculated: 9.2%
      Light Sleep Pct Calculated: 71.8%
      Deep Sleep Pct Calculated: 7.2%
```

- **Basic clustering (all nights):**
  - Nights are grouped using features like sleep score, duration, efficiency, and bedtime hour.
  - Each cluster represents a **recurring sleep pattern**, e.g.:
    - Shorter sleep & lower score.
    - Moderate duration & mid score.
    - Longer sleep & higher score.
- **Enhanced clustering (434 nights with stages):**
  - Adds REM/deep percentages and other stage metrics.
  - You get richer “profiles” like:
    - Pattern with higher REM percentage and good scores.
    - Pattern with low REM or fragmented sleep and worse scores.

EXECUTIVE SUMMARY

In [ ]:
```python
if 'overall_sleep_score' in df.columns:
    total_nights = len(df)
    avg_score = df['overall_sleep_score'].mean()
    score_std = df['overall_sleep_score'].std()

    print(f"DATASET OVERVIEW:")
    print(f"   • Total sleep records analyzed: {total_nights:,}")
    if 'sleep_date' in df.columns:
        print(f"   • Data collection period: {df['sleep_date'].min()} to {df['sleep_date'].max()}")
    print(f"   • Average sleep quality: {avg_score:.1f} ± {score_std:.1f}")

    # Data completeness summary
```

```
    if 'has_complete_stage_data' in df.columns:
        complete_stage_nights = df['has_complete_stage_data'].sum()
        print(f"   • Complete sleep stage data: {complete_stage_nights}/{total_nights} nights ({complete_stage_

    # Performance categories
    excellent_nights = (df['overall_sleep_score'] >= 85).sum()
    good_nights = ((df['overall_sleep_score'] >= 70) & (df['overall_sleep_score'] < 85)).sum()
    fair_nights = ((df['overall_sleep_score'] >= 60) & (df['overall_sleep_score'] < 70)).sum()
    poor_nights = (df['overall_sleep_score'] < 60).sum()

    print(f"\nSLEEP QUALITY BREAKDOWN:")
    print(f"   • Excellent nights (85-100): {excellent_nights} ({excellent_nights/total_nights*100:.1f}%)")
    print(f"   • Good nights (70-84): {good_nights} ({good_nights/total_nights*100:.1f}%)")
    print(f"   • Fair nights (60-69): {fair_nights} ({fair_nights/total_nights*100:.1f}%)")
    print(f"   • Poor nights (<60): {poor_nights} ({poor_nights/total_nights*100:.1f}%)")

# Key metrics summary
key_metrics_summary = {}
if 'sleep_duration_hours' in df.columns:
    key_metrics_summary['Average Sleep Duration'] = f"{df['sleep_duration_hours'].mean():.1f} hours"
if 'sleep_efficiency_percent' in df.columns:
    key_metrics_summary['Average Sleep Efficiency'] = f"{df['sleep_efficiency_percent'].mean():.1f}%"
if 'bedtime_hour' in df.columns:
    key_metrics_summary['Most Common Bedtime'] = f"{df['bedtime_hour'].mode().iloc[0]}:00"

print(f"\nKEY PERFORMANCE INDICATORS:")
for metric, value in key_metrics_summary.items():
    print(f"   • {metric}: {value}")
```

```
DATASET OVERVIEW:
   • Total sleep records analyzed: 818
   • Data collection period: 2023-08-05 to 2025-08-08
   • Average sleep quality: 71.0 ± 13.9
   • Complete sleep stage data: 434/818 nights (53.1%)

SLEEP QUALITY BREAKDOWN:
   • Excellent nights (85-100): 80 (9.8%)
   • Good nights (70-84): 471 (57.6%)
   • Fair nights (60-69): 103 (12.6%)
   • Poor nights (<60): 164 (20.0%)

KEY PERFORMANCE INDICATORS:
   • Average Sleep Duration: 7.0 hours
   • Average Sleep Efficiency: 85.4%
   • Most Common Bedtime: 23:00
```

FINAL INSIGHTS, CONCULSIONS AND RECOMMENDATIONS

Core Findings from the Data

1. My sleep quality is decent but volatile

I maintain an average sleep score of 71/100, but the standard deviation of ~14 shows large variation. This tells me I don't struggle with chronic poor sleep, but I lack stability. Some nights are excellent, but enough bad nights exist to drag my overall baseline down.

I don't have a sleep disorder pattern — I have a discipline and consistency problem.

---

2. Sleep duration is the strongest driver of my sleep quality

The correlation between sleep duration and sleep score is extremely high ($r \approx 0.739$). This makes it clear that when I sleep longer, I almost always sleep better.

No fancy metric matters if I don't spend enough time asleep. This is the single most important lever I control.

---

3. Sleep debt is a serious issue for me

I am in sleep debt on 70.2% of nights.
My average nightly debt is 1 hour.

Even worse, the correlation between sleep debt and sleep quality is -0.739, which is a strong negative relationship. This means I'm consistently paying a performance tax for my short nights.

I am not "occasionally cutting sleep" — I am structurally under-sleeping.

---

4. Efficiency is good but not compensating for duration

My average sleep efficiency is around 85%, which is healthy. This means that when I'm in bed, I generally sleep well. However, efficiency is not saving me from the damage caused by insufficient total sleep.

In simple terms: I fall asleep well, but I don't stay asleep long enough.

---

5. My REM sleep is below optimal

On nights where stage data is available, my average REM percentage is 17.3%, while the healthy range is 20–25%.

There is a strong positive correlation ($r \approx 0.658$) between REM percentage and sleep score. This tells me that improving REM sleep is a direct path to better cognitive recovery, memory consolidation, and mental sharpness.

This likely explains days where I feel physically rested but mentally foggy.

---

6. Weekdays vs weekends make no real difference

Statistical testing shows no meaningful difference between my weekend and weekday sleep scores. This means I maintain a relatively consistent schedule — which is good — but also means I'm not using weekends to recover properly.

I am stable, but I am stably under-sleeping.

---

7. Seasons affect my sleep quality

There is a statistically significant difference in sleep quality across seasons. I sleep best during winter and summer and worst during autumn. Cooler temperatures are clearly associated with higher sleep scores.

Temperature is not a minor factor for me — it is a performance variable.

---

8. My best sleep pattern is clear

From clustering analysis, my highest-quality sleep profile includes:

- Sleep duration ~7.8 hours
- Sleep efficiency ~87%
- REM ~21%
- Earlier bedtime
- Sleep score ~81+

This pattern proves that my body is fully capable of elite sleep — I just don't consistently create the conditions for it.

High-Impact Personal Conclusions

- I am not a "bad sleeper" — I am a chronically under-rested one.
- My main enemy is not efficiency or sleep cycles — it is insufficient duration.
- Most of my bad nights are self-inflicted through late bedtimes and poor sleep prioritisation.
- When I sleep properly, my metrics automatically improve across the board.
- My REM sleep needs targeted improvement for better mental recovery.

This is not a medical problem. This is a behavioural optimisation problem.

Actionable Recommendations

1. Non-negotiable: Increase sleep duration

I need to shift my target from 7 hours to at least 8–8.5 hours consistently. My data clearly proves that duration drives quality more than anything else.

No shortcuts. No hacks. Just time asleep.

---

2. Fix sleep debt aggressively

I should schedule deliberate recovery nights where I sleep 8.5–9 hours to offset accumulated deficits. Treating sleep debt like financial debt is a mistake — biological debt compounds.

---

3. Optimise REM sleep

To improve REM percentage, I should:

- Avoid screens 60–90 minutes before bed
- Reduce late-night caffeine
- Maintain a cooler sleeping environment
- Prioritise consistent bedtime windows

Increasing REM is directly linked to higher sleep scores and better next-day cognition.

---

4. Lock in bedtime consistency

My most common bedtime is 23:00. I should treat this as my anchor time and build strict discipline around it rather than drifting even later.

Consistency will stabilise not just sleep time, but sleep depth.

---

5. Leverage temperature strategically

Colder nights correlate with better sleep for me. I should consciously optimise room temperature, ventilation, or seasonal adjustments instead of passively accepting suboptimal conditions.

---

6. Monitor trends, not individual nights

Single bad nights aren’t the problem. Patterns are. I should focus on rolling 7-day averages instead of emotionally reacting to one poor score.